

ROLE: Full Stack Developer

OVERVIEW

Full Stack Developers design, develop, and maintain both frontend and backend components of web applications. They work across the entire software stack – UI, APIs, databases, and deployment – ensuring applications are scalable, secure, and performant. A full stack developer is expected to own a feature end to end, from the button a user clicks to the database query that serves the data.

CORE RESPONSIBILITIES

- Design, develop, and maintain full-stack web applications
- Build and integrate RESTful APIs and third-party services
- Develop responsive and interactive frontend interfaces
- Implement backend services and business logic
- Manage database design and data flow
- Optimize applications for performance, scalability, and security
- Collaborate with designers, backend developers, and product teams
- Write clean, reusable, and maintainable code
- Perform debugging, testing, and code reviews
- Participate in system design and architecture discussions

REQUIRED PROGRAMMING SKILLS

Every full stack developer must be strong in JavaScript (ES6+) and TypeScript since they power both frontend and backend (Node.js). Python is the most common second language for backend work. HTML5 and CSS3 are non-negotiable for frontend. Git for version control is mandatory from day one. Understanding Object-Oriented Programming and basic Data Structures is expected even at entry level.

- JavaScript (ES6+)
- TypeScript
- Python or PHP for backend
- HTML5 and CSS3
- Object-Oriented Programming
- Version Control with Git
- Basic Data Structures

FRONTEND DEVELOPMENT

The frontend is everything the user sees and interacts with. React.js is the most in-demand frontend framework in 2024-25, followed by Vue.js. Angular is used mostly in enterprise. Tailwind CSS has largely replaced Bootstrap for styling. State management with Redux or Context API is expected for larger apps. All frontend code must be responsive – mobile-first design is standard.

- React.js (most in demand)
- Vue.js
- Angular (enterprise)
- Tailwind CSS and Bootstrap
- Material UI
- Responsive Web Design
- State Management using Redux or Context API

BACKEND DEVELOPMENT

The backend handles business logic, authentication, and data processing. Node.js with Express is the most common backend for JavaScript developers. Python developers use Django for full-featured apps or FastAPI for high-performance APIs. RESTful API design is core – every backend developer must know how to design clean, versioned APIs. Authentication using JWT and OAuth is expected at intermediate level.

- Node.js and Express
- Django and Flask for Python
- FastAPI for high-performance APIs
- Laravel for PHP
- RESTful API Design
- Authentication using JWT and OAuth

DATABASE SKILLS

Databases are where all application data lives. PostgreSQL and MySQL are the standard relational databases – you must be comfortable writing complex SQL queries, designing schemas, and understanding indexing. MongoDB is the go-to for document-based NoSQL storage. Database design skills (normalization, relationships) are tested in interviews. Query optimization matters at scale.

- MySQL and PostgreSQL for relational data
- MongoDB for NoSQL document storage
- Database Design and Schema Normalization
- Query Optimization and Indexing
- Understanding of ACID properties

DEVOPS AND DEPLOYMENT

Full stack developers are expected to deploy their own applications, not just write code. Docker is essential – containerization is standard practice. CI/CD pipelines with GitHub Actions automate testing and deployment. Cloud platforms (AWS, GCP, Azure) host production applications. Basic Kubernetes knowledge is a plus at intermediate to advanced level.

- Docker for containerization
- GitHub Actions for CI/CD pipelines
- AWS, GCP, or Azure for cloud hosting
- Basic Kubernetes for orchestration
- Environment configuration and secrets management

SYSTEM DESIGN AND ENGINEERING FUNDAMENTALS

At intermediate and above, full stack developers participate in system design discussions. This includes understanding how to design scalable systems, where to use caching (Redis), how load balancers work, and security best practices like input validation and HTTPS. Software design patterns (MVC, Repository pattern) are applied daily.

- Scalability concepts and horizontal vs vertical scaling
- Caching strategies using Redis
- Software Design Patterns like MVC and Repository
- Security best practices including HTTPS, input validation, rate limiting
- Load handling and performance optimization

API AND INTEGRATION SKILLS

Modern web apps are built by integrating multiple APIs and services. REST API development and consumption is core. AJAX and the Fetch API are used for asynchronous frontend-backend communication. Webhooks enable event-driven integrations. JSON is the standard data format for API communication.

- REST API development and consumption
- AJAX and Fetch API for async communication
- Third-party API integration
- Webhooks for event-driven systems
- JSON for data serialization

SOFTWARE QUALITY PRACTICES

Writing code that works once is not enough – it must be testable and maintainable. Unit testing and integration testing catch bugs early. Code reviews improve team code quality. CI pipelines run tests automatically on every commit. Documentation ensures others (and future you) can understand the codebase.

- Unit Testing and Integration Testing
- Code Reviews as standard practice
- Continuous Integration with automated test runs
- Debugging practices and systematic troubleshooting
- Code documentation standards

TOOLS AND COLLABORATION

- Git, GitHub, GitLab for version control and collaboration
- Jira for project and issue tracking
- Agile and Scrum methodology for team workflow

- Confluence for documentation

ENTRY LEVEL EXPECTATIONS

At entry level, you are expected to build simple full-stack applications independently. The minimum bar is: HTML, CSS, JavaScript basics, one frontend framework (preferably React), one backend framework, basic REST API understanding, basic SQL queries, and Git. A portfolio project demonstrating these is more valuable than certificates.

- HTML, CSS, JavaScript basics
- One frontend framework (React preferred)
- One backend framework (Node/Express or Django)
- Basic REST API understanding
- Basic SQL
- Git workflow

INTERMEDIATE LEVEL EXPECTATIONS

At intermediate level, you have shipped features in a real project or built non-trivial portfolio projects. You can integrate third-party APIs, implement authentication systems, deploy applications to the cloud, optimize database queries, and use Docker. You understand the full request-response lifecycle.

- Full-stack project experience end to end
- API integration and authentication systems
- Deployment experience on cloud platforms
- Database optimization
- Docker basics

ADVANCED LEVEL EXPECTATIONS

Advanced full stack developers design systems, not just implement them. They make architectural decisions, set up CI/CD pipelines, deploy microservices, handle performance bottlenecks, and enforce security practices across the codebase.

- Scalable system design
- CI/CD pipeline setup and maintenance
- Cloud deployment and infrastructure
- Microservices architecture
- Performance tuning and security hardening

COMMON PROJECTS FOR PORTFOLIO

These project types are commonly built and demonstrate full-stack competence to employers. A well-built e-commerce app or SaaS dashboard covers almost every skill area listed above.

- E-commerce platforms with cart, payment, and admin panel
- Social media applications with real-time features
- CRM and dashboard systems
- SaaS multi-tenant applications
- Job portals
- Blogging platforms with rich text editor
- Admin dashboards with analytics
- API-based web apps consuming public APIs

RELATED ROLES

- Frontend Developer
- Backend Developer
- Software Engineer
- DevOps Engineer
- Web Developer
- Application Developer